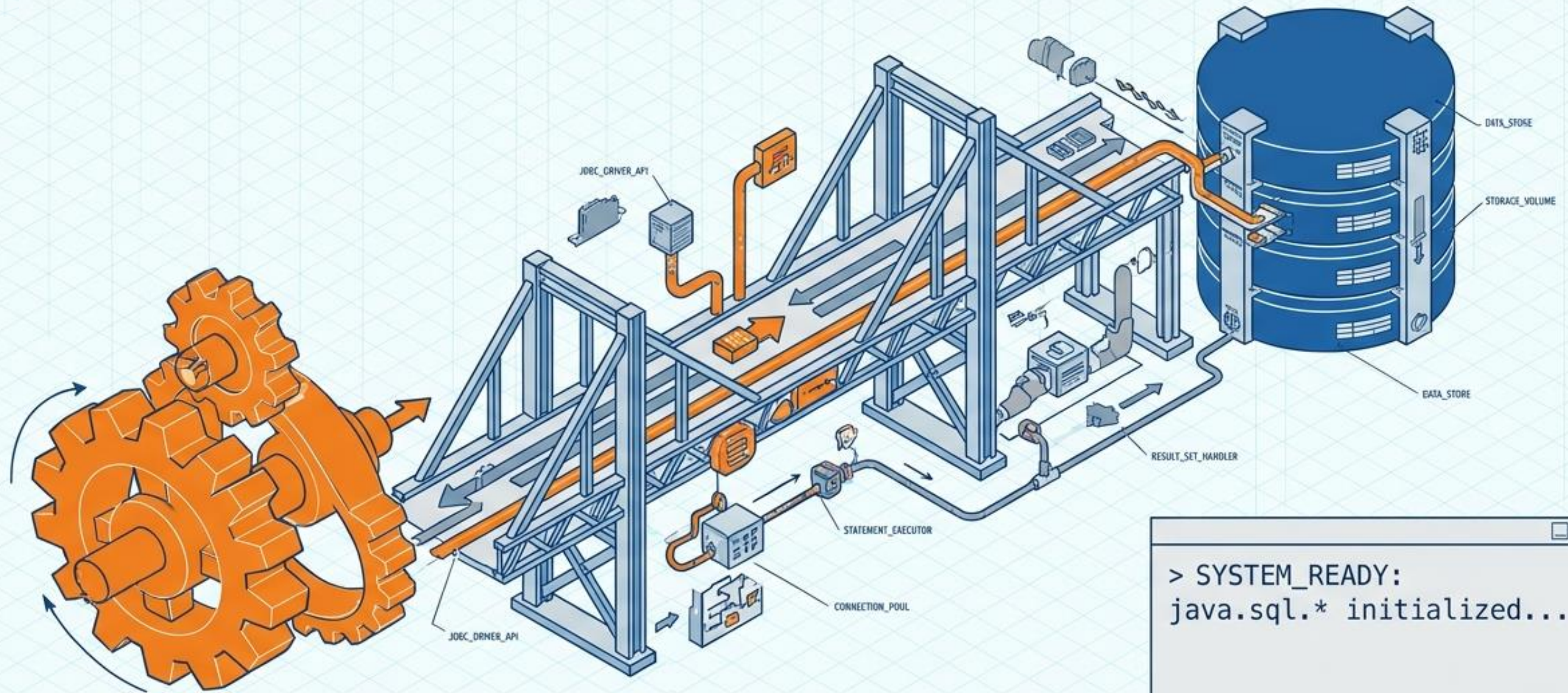


Bridging Two Worlds: The Java Database Connectivity Blueprint

Building the Industrial Infrastructure for Software Data Access



```
> SYSTEM_READY:  
java.sql.* initialized...
```


The Communication Gap

The Universal Translator Gearbox

Java Application

Objects & Classes

JDBC API

Relational DBMS

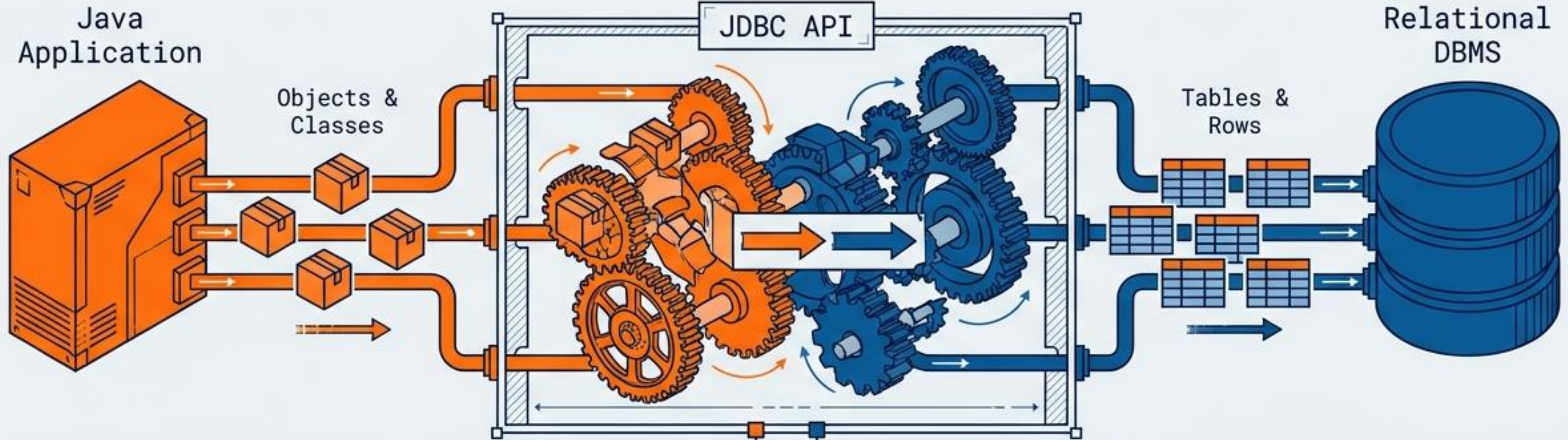
Tables & Rows

The Problem

Java applications and relational databases speak fundamentally incompatible languages. Objects cannot map natively to flat tables without translation.

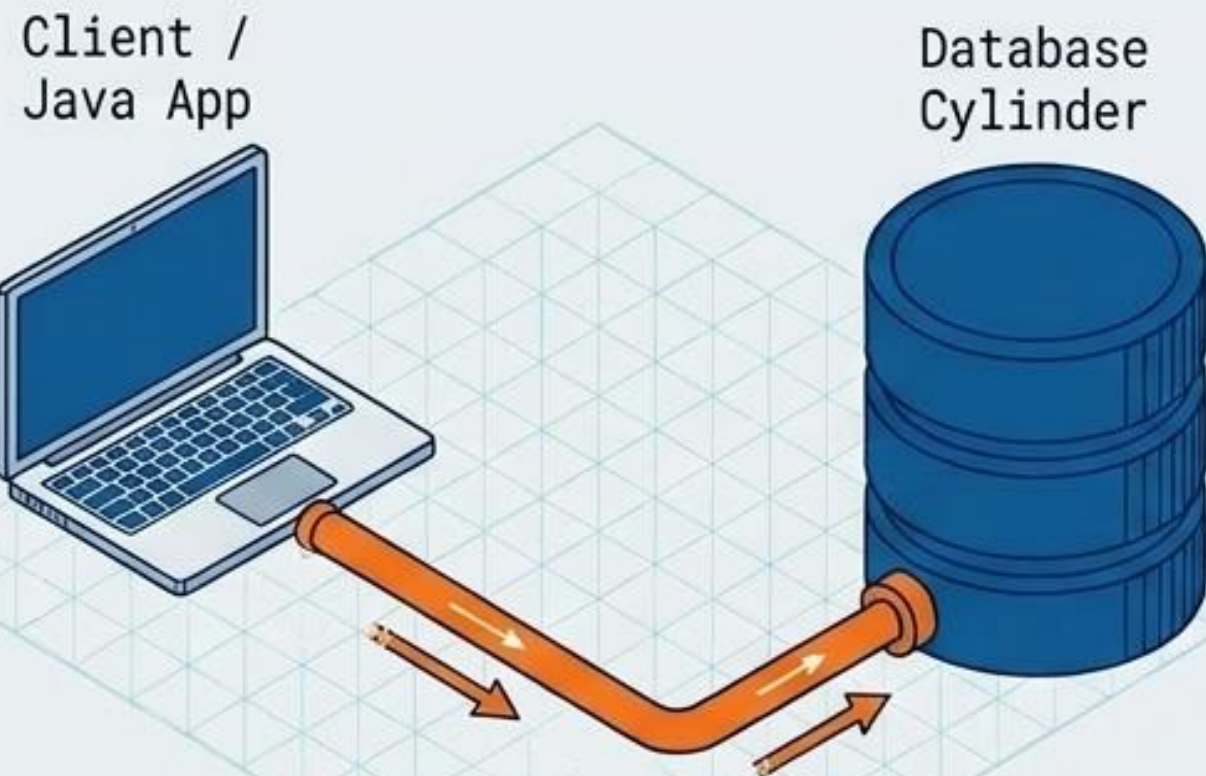
The Solution

JDBC acts as a client-side mechanical adapter, catching Java requests and translating them into a network protocol the DBMS natively understands.



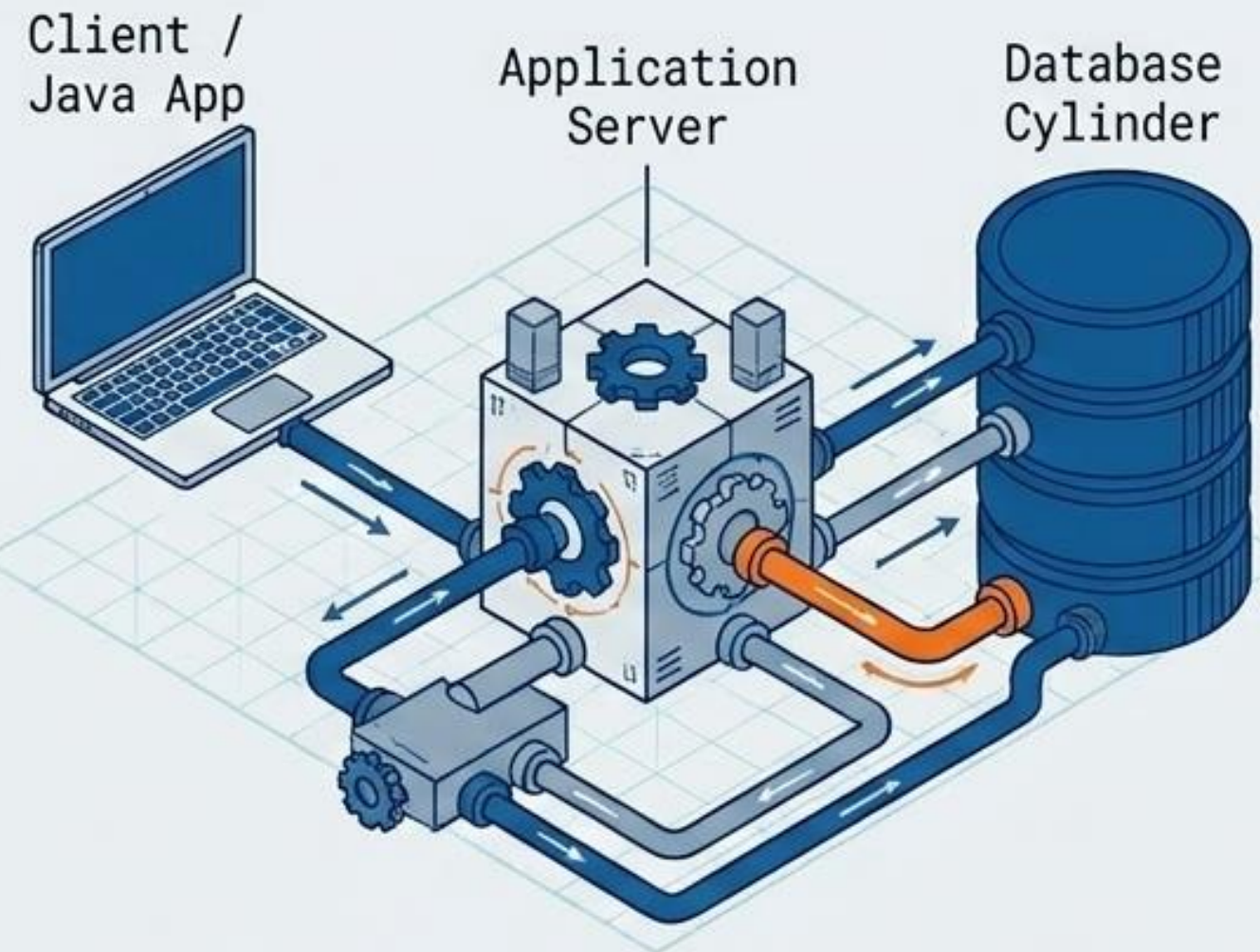
Architecture Matrix: Two-Tier vs. Three-Tier Models

Two-Tier Architecture



- **Structure:** Client → Database
- **JDBC Driver Location:** Client Machine
- **Best For:** Direct client/server setups and simple internal applications.

Three-Tier Architecture



- **Structure:** Client → App Server → Database
- **JDBC Driver Location:** Middle-Tier Service
- **Best For:** Scalable enterprise applications requiring connection pooling and advanced security.

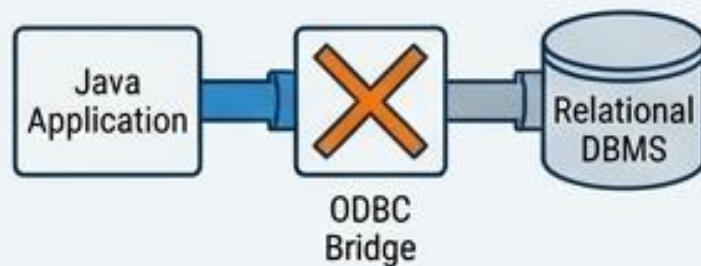
The Driver Evolution Matrix

◦ Technological Progression ◦

Type 1: JDBC-ODBC Bridge

Status: Deprecated

Requires local Microsoft ODBC setup. No longer supported in modern Java.



Type 2: Native-API

Status: Legacy

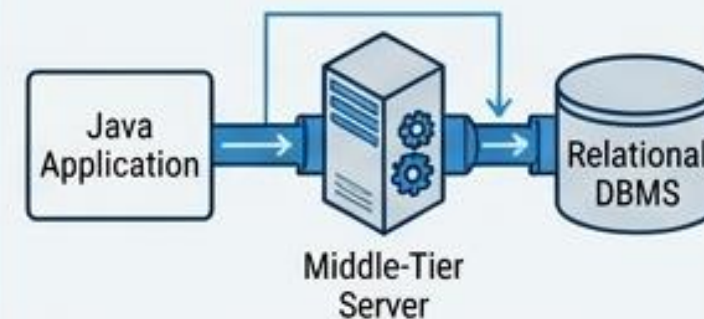
Partially Java. Requires client-side database libraries to be physically installed.



Type 3: Network Protocol

Status: Specialized

Fully Java. Translates JDBC to a middle-tier server protocol rather than direct DB access.



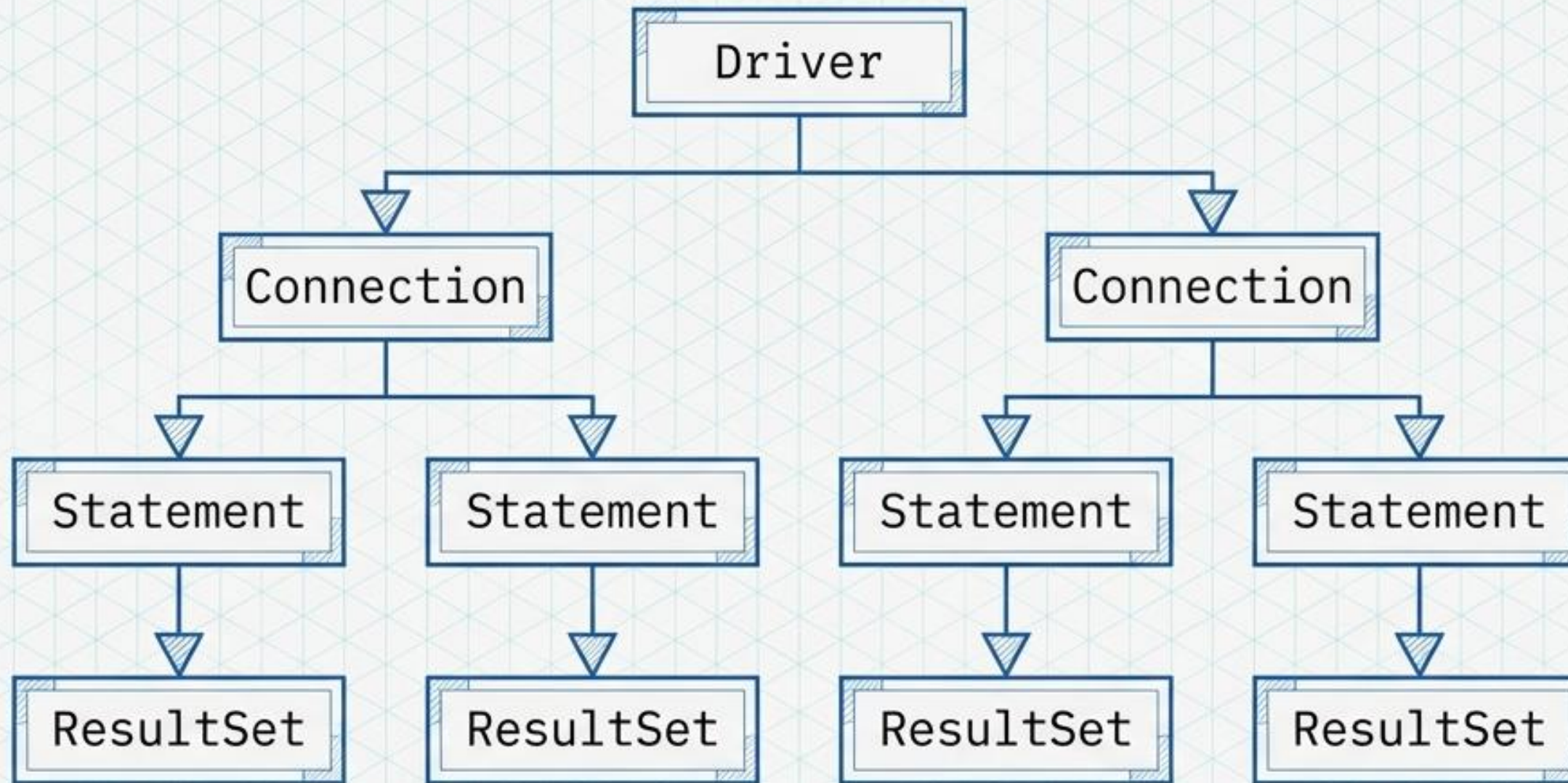
Type 4: Thin Driver

Status: The Modern Standard

Fully Java. Converts JDBC calls directly into the vendor-specific database protocol (e.g., MySQL, Oracle). Requires no native client libraries.



The Component Hierarchy Blueprint



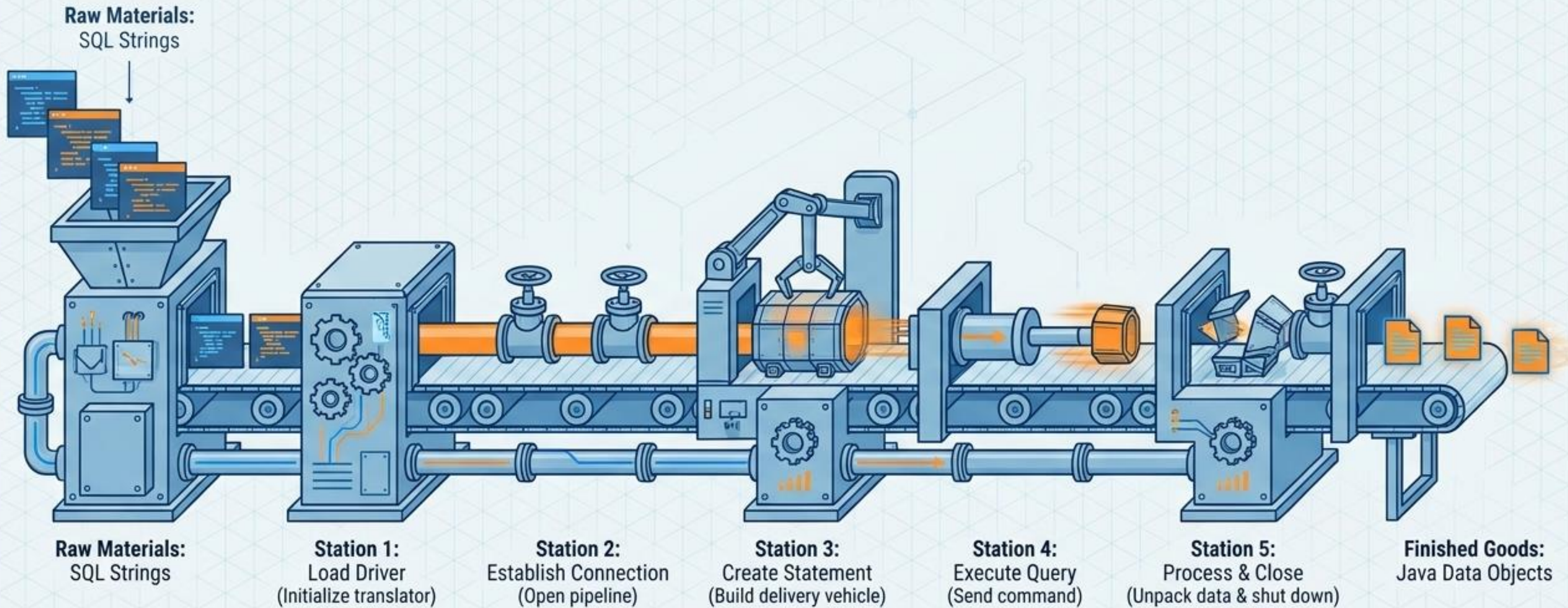
The Dependency Chain

Every database operation flows strictly down this architectural tree. You must acquire the parent object before instantiating the child. A **ResultSet** cannot exist without a **Statement**, and a **Statement** cannot exist without a **Connection**.

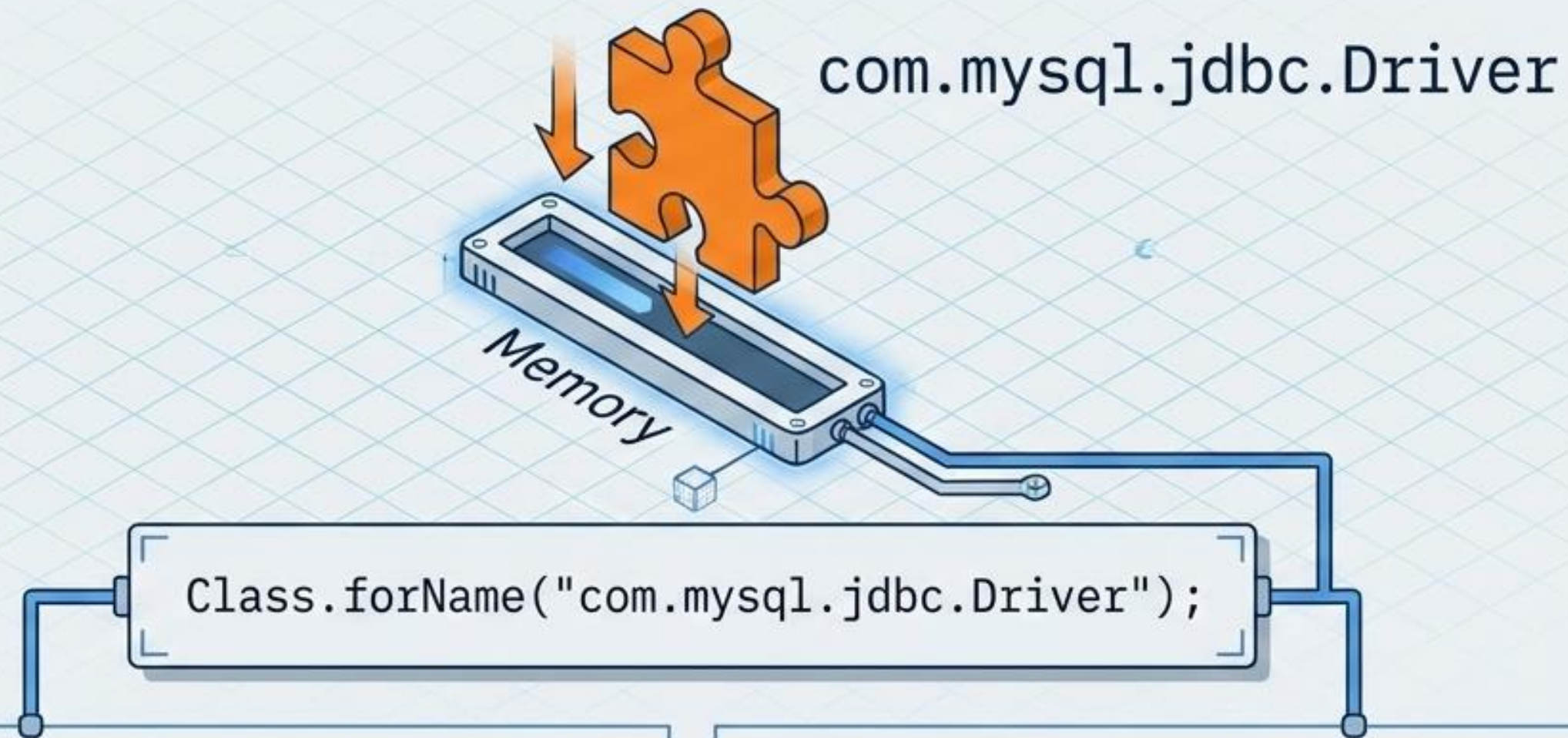
SQL

- To access or write applications for database systems, you need to use the Structured Query Language (SQL).
- SQL is the universal language for accessing relational database systems.
- Application programs may allow users to access database without directly using SQL, but these applications themselves must use SQL to access the database.
- Sql statements are used to create tables, insert record, update records, query database, delete records delete tables,.....

The Assembly Line: Executing a Query



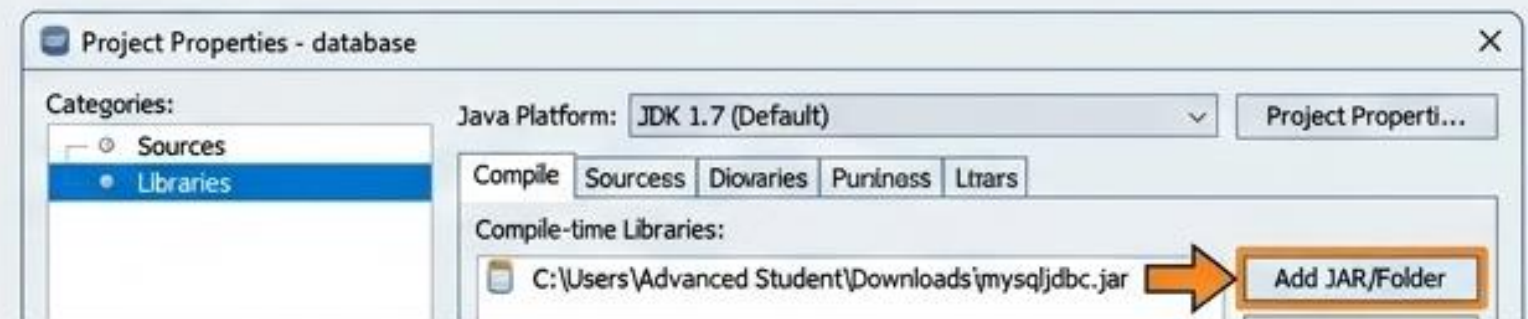
Station 1: Loading the Driver



Blueprint Notes

- Action: Loads the driver's specific class file into client memory at runtime.
- Modern Context: Since JDBC 4.0, drivers are auto-loaded if present, making manual registration largely optional.

Prerequisite: The Classpath



The Connector JAR must be physically added to the project build path.

Project Properties - database

Categories:

- Sources
- Libraries
- Build
 - Compiling
 - Packaging
 - Documenting
- Run
- Application
 - Web Start
- Formatting

Java Platform: JDK 1.7 (Default)

Manage Platforms...

Libraries Folder:

Browse...

Compile Processor Run Compile Tests Run Tests

Compile-time Libraries:

C:\Users\Advanced Student\Downloads\mysqljdbc.jar

Add Project...

Add Library...

Add JAR/Folder

Edit

Remove

Move Up

Move Down

Compile-time libraries are propagated to all library categories.

☒ Build Projects on Classpath

OK

Cancel

Help

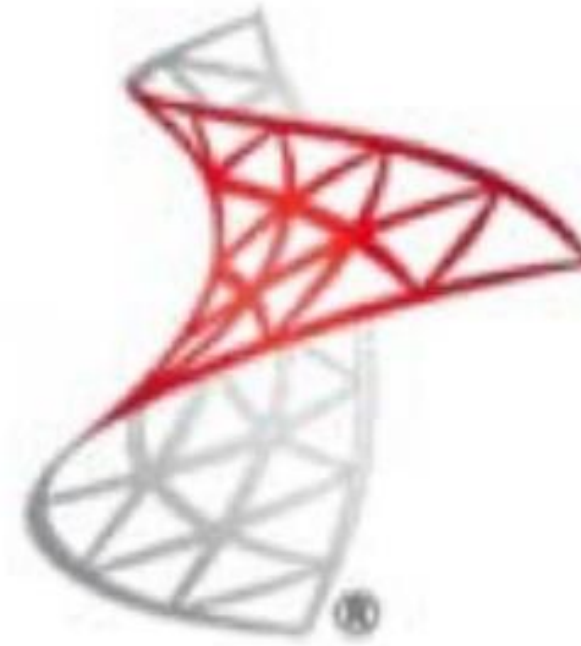
VENDOR SPECIFICITY

JDBC drivers are **database-specific**. You cannot use an Oracle driver to connect to a MySQL database.

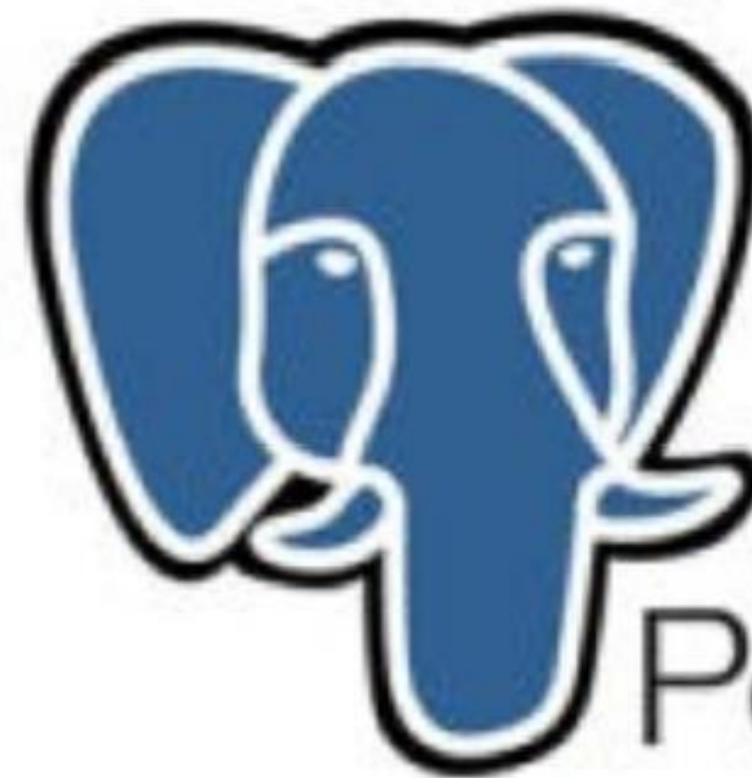
Every major database vendor provides their own proprietary driver class name that must be loaded via

```
Class.forName()
```

These drivers translate standard Java JDBC calls into the specific protocol understood by the database server.



Microsoft®
SQL Server



PostgreSQL

OR

DRIVER REGISTRY OVERVIEW

Database	Fully Qualified Driver Class Name	Connection URL Format
MySQL	<code>com.mysql.cj.jdbc.Driver</code>	<code>jdbc:mysql://host:port/db</code>
Oracle	<code>oracle.jdbc.driver.OracleDriver</code>	<code>jdbc:oracle:thin:@host:port:sid</code>
PostgreSQL	<code>org.postgresql.Driver</code>	<code>jdbc:postgresql://host:port/db</code>
SQL Server	<code>com.microsoft.sqlserver.jdbc.SQLServerDriver</code>	<code>jdbc:sqlserver://host:port;dbName=...</code>

Example : a method to load drivers

```
public static void LoadDriver(){  
try{  
//Load Driver for MS Access  
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");  
//Load Driver for MySQL  
Class.forName("com.mysql.jdbc.Driver");  
//Load Driver for Oracle  
Class.forName("oracle.jdbc.driver.OracleDriver");  
}  
catch (ClassNotFoundException ex){  
System.out.println("DataBase driver can not be loaded");  
return;  
}  
System.out.println("DataBase driver loaded successfully ");  
}
```


SAFETY & EXCEPTION HANDLING

The Try-Catch Block

`Class.forName()` throws a checked exception: **ClassNotFoundException**. This must be handled to prevent application crashes.

```
try { Class.forName("..."); } catch  
(ClassNotFoundException e) { e.printStackTrace(); }
```

Failure Scenarios

Common reasons for failure include:

- The **JAR file** is missing from the classpath.
- Typo in the **Driver String** name.
- Incompatible **JDK version** for the provided driver version.

Station 2: Establishing the Connection



```
Connection con = DriverManager.getConnection(url, user, password);
```

`jdbc:mysql://localhost:3306/testdb`

The API
Protocol

The Sub-protocol
(Database type)

The Hostname
and Port

The Target
Database Name

Examples:

For Access:

```
Connection connection = DriverManager.getConnection  
("jdbc:odbc:ExampleMDBDataSource");
```

For MySQL:

```
Connection connection = DriverManager.getConnection  
("jdbc:mysql://localhost/test","scott", "tiger");
```

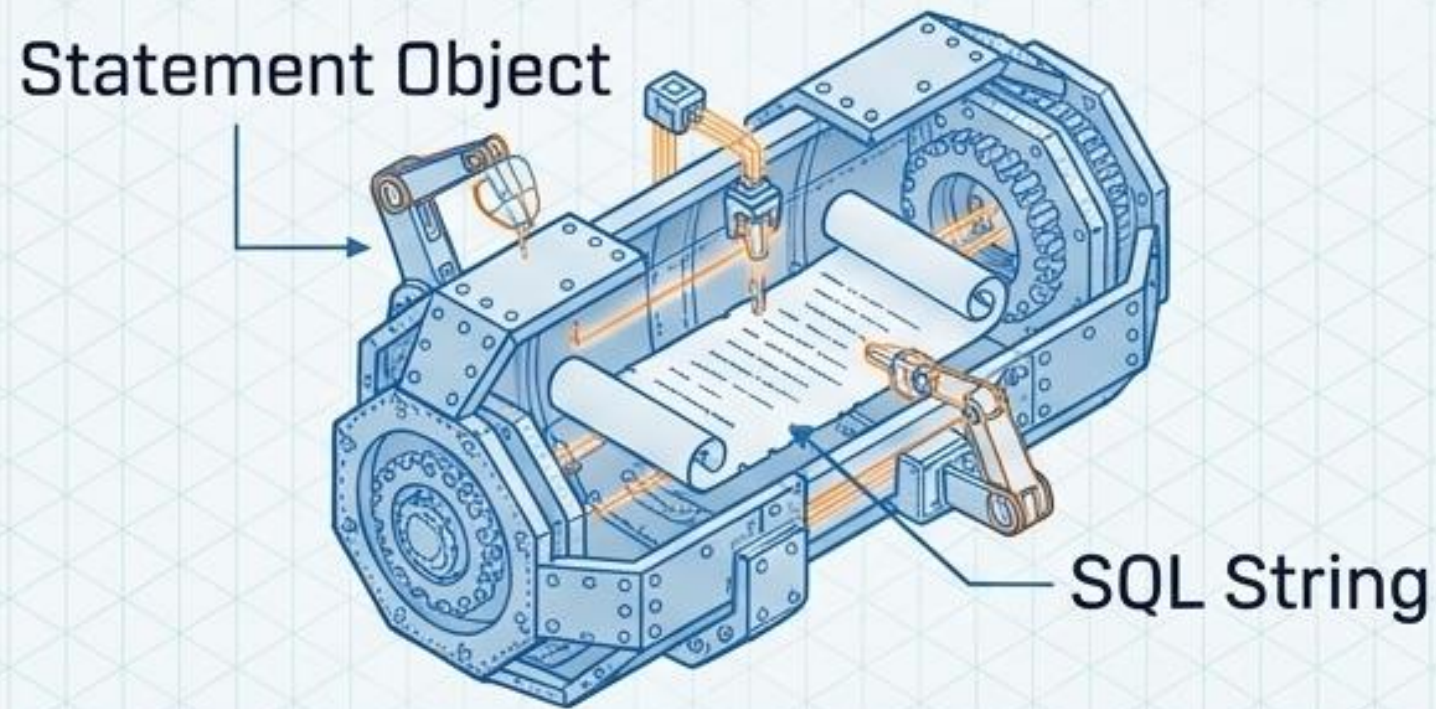
For Oracle:

```
Connection connection = DriverManager.getConnection  
("jdbc:oracle:thin:@liang.armstrong.edu:1521:orcl", "scott", "tiger");
```



```
public static void connect(){
try{
Connection connection =
DriverManager.getConnection("jdbc:mysql://localhost/aast","root", "");
}
catch(SQLException ex) {
System.out.println("Can not connect to database");
return;
}
System.out.println(" Connected to DataBase successfully ");
}
```


Station 3: Creating the Statement

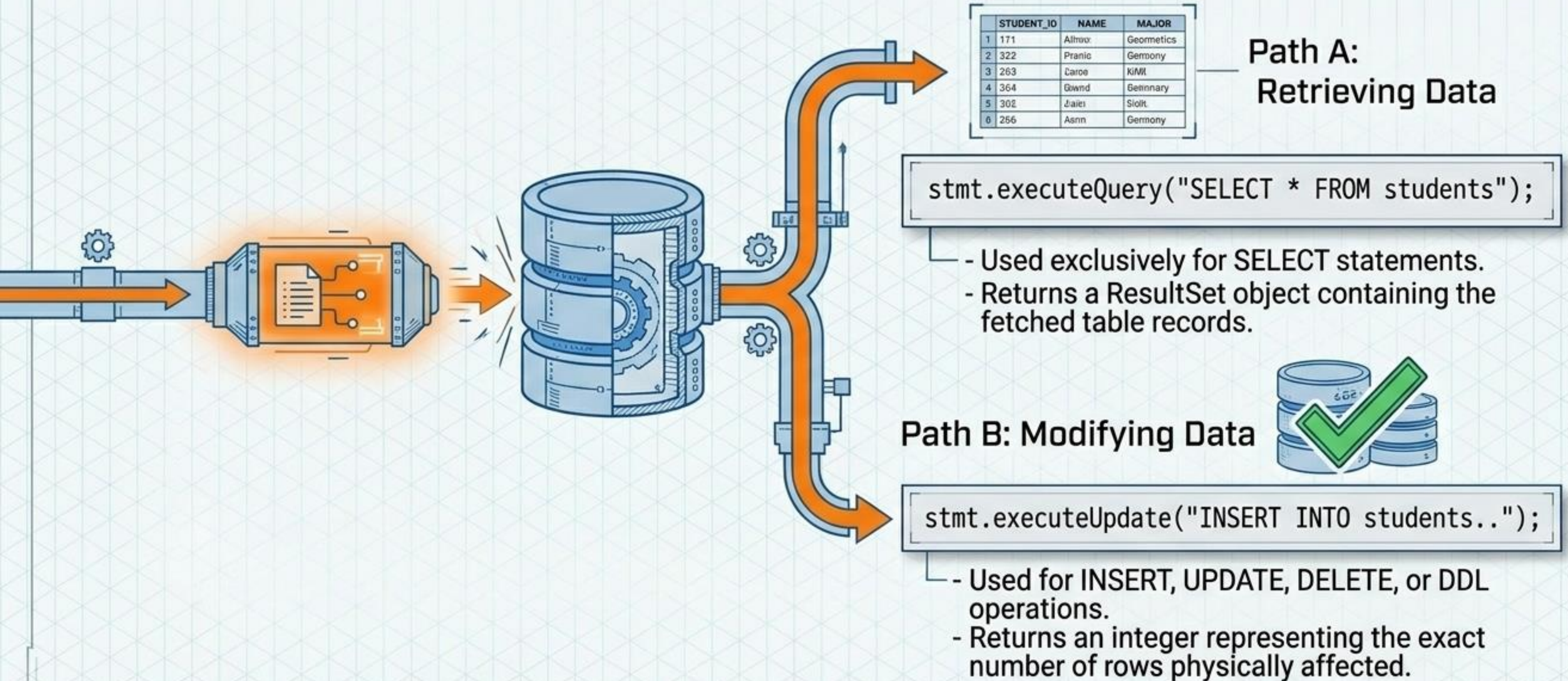


```
Statement stmt = con.createStatement();
```

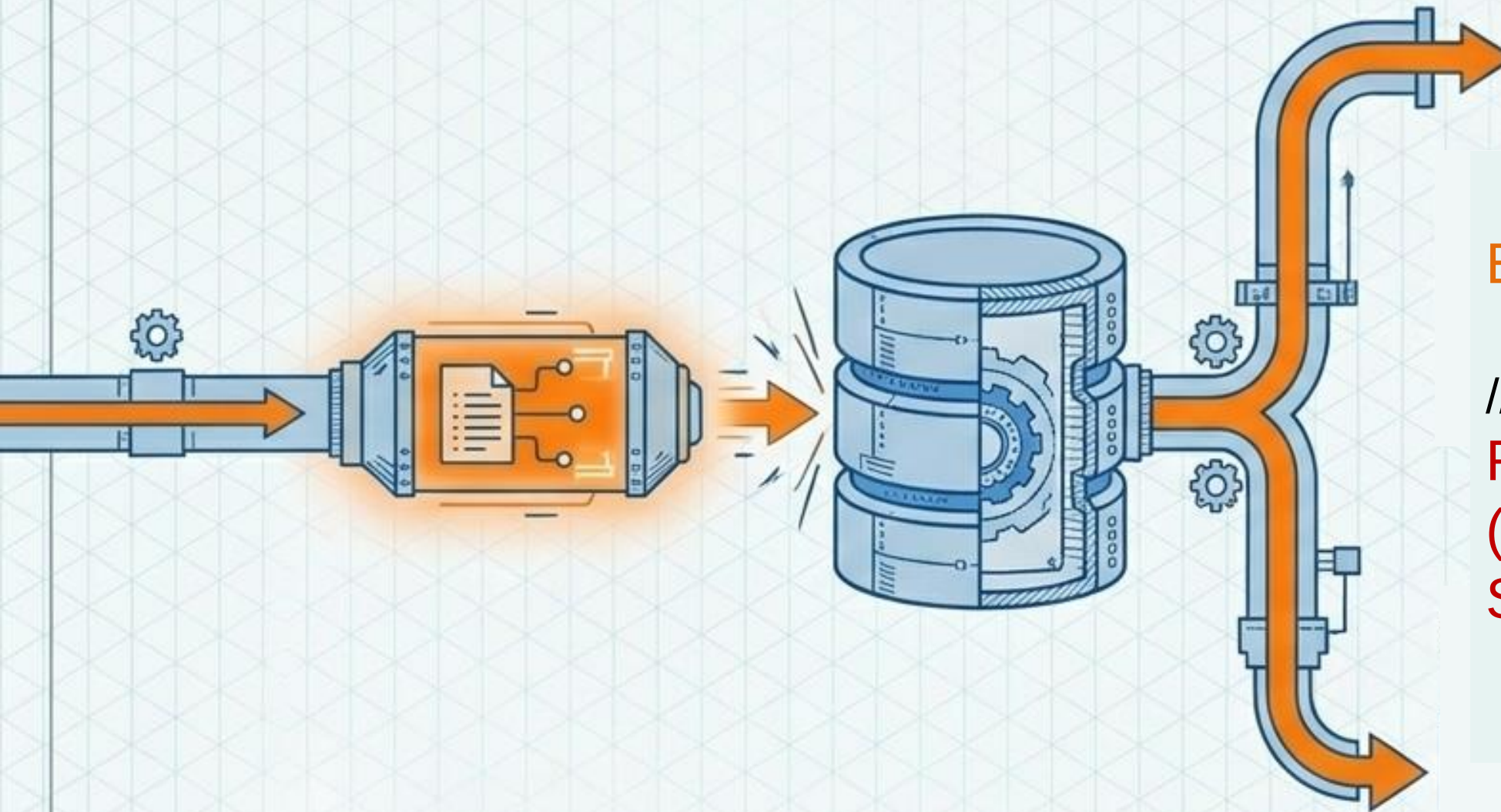
The Statement Interface Matrix

	Interface	Description
1.	Statement	For executing simple, static SQL queries without parameters.
2.	PreparedStatement	Precompiled SQL. Used for dynamic queries with parameters. Best Practice for real-world applications to prevent SQL injection.
3.	CallableStatement	Used strictly to execute specialized stored procedures residing within the database.

Station 4: Executing the SQL



Station 4: Executing the SQL



Executing statement (for select):

```
// Select the columns from the Student table  
ResultSet rs = stmt.executeQuery  
("select firstName, mi, lastName from  
Student where lastName" + " = 'Smith'");
```


Station 5a: Processing the ResultSet

The Data Cursor Mechanism

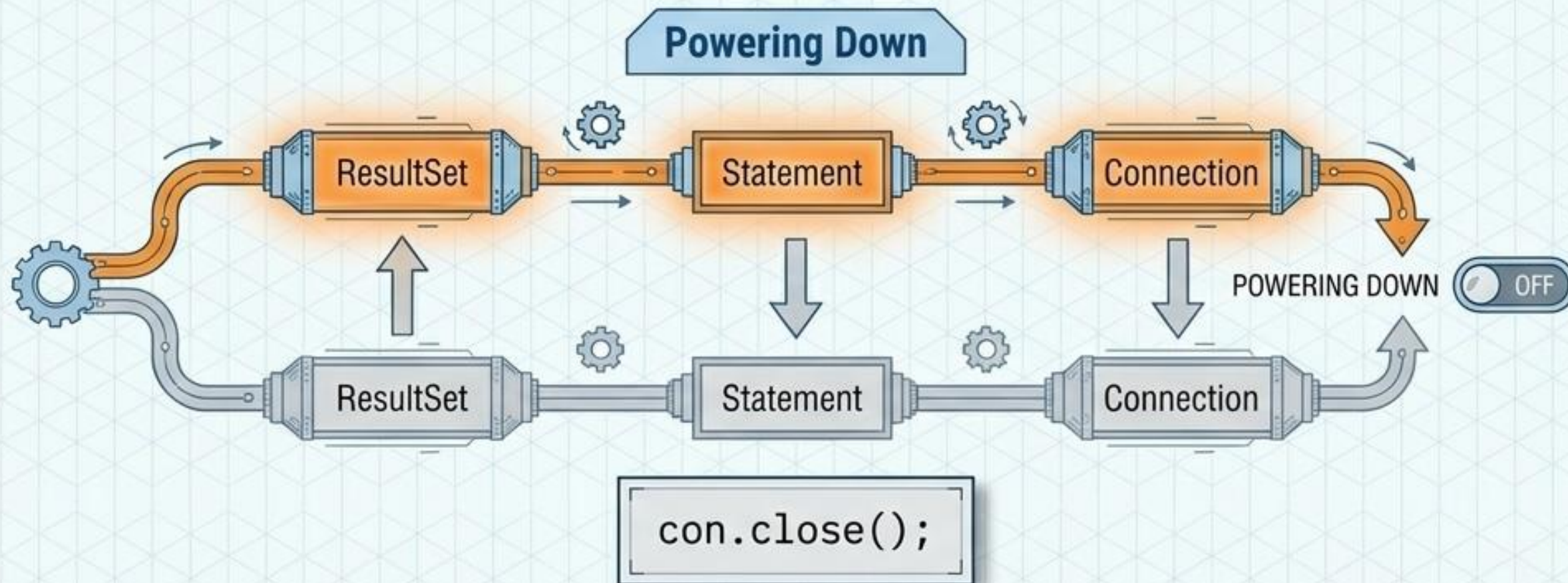
Cursor Point

registration	Name	Grade
11234	Ahmed Aly	A
10234	Mohamed Tarek	B
10023	Fathi Ahmed	F

```
while (rs.next()) {  
    System.out.println(rs.getString("Name"));  
}
```

- 1. Iteration:** `rs.next()` moves the cursor forward exactly one row from its current position. It returns `false` when no rows remain, breaking the while loop.
- 2. Extraction:** `getString(columnLabel)` physically extracts the specific datatype from the currently active row. Column indexing in JDBC begins at 1, not 0.

Station 5b: Teardown & Connection Closure



Best Practice Warning: Resource Management

- **Automatic Teardown:** Manually closing the parent `Connection` object automatically shuts down its dependent `Statement` and `ResultSet` objects.
- **Memory Leaks:** Failing to close connections will result in catastrophic memory leaks and locked database tables on the server side.
- **The Modern Standard:** Always wrap database operations in a Java "try-with-resources" block to guarantee automatic closure, even if an SQL exception is thrown during execution.


```
import java.sql.*;
public class SimpleJdbc {
    public static void main(String[] args)
        throws SQLException, ClassNotFoundException {
// Load the JDBC driver
        Class.forName("com.mysql.jdbc.Driver");
        System.out.println("Driver loaded");
// Establish a connection
        Connection connection =
            DriverManager.getConnection("jdbc:mysql://localhost/aast","root", "");
        System.out.println("Database connected");
// Create a statement
        Statement statement = connection.createStatement();
// Execute a statement
        ResultSet resultSet = statement.executeQuery
            ("select Name,Grade from Students where registration="+435234);
// Iterate through the result and print the student names
        while (resultSet.next())
            System.out.println( " Student :" +resultSet.getString(1) + "\t" +
                "grade:" +resultSet.getString(2));
// Close the connection
        connection.close();
    }}
```


Assignment: Student Database

User Interface

Student Registration

Connect Registration Number 10234 Get Grade Clear

Number	Name	Grade
10234	Mohamed Tarek	B

1. UI passes
Registration
Input: 10234

Java Application & JDBC Translator

2. App formats
dynamic SQL string.

3. JDBC driver
translates protocol and
executes statement.

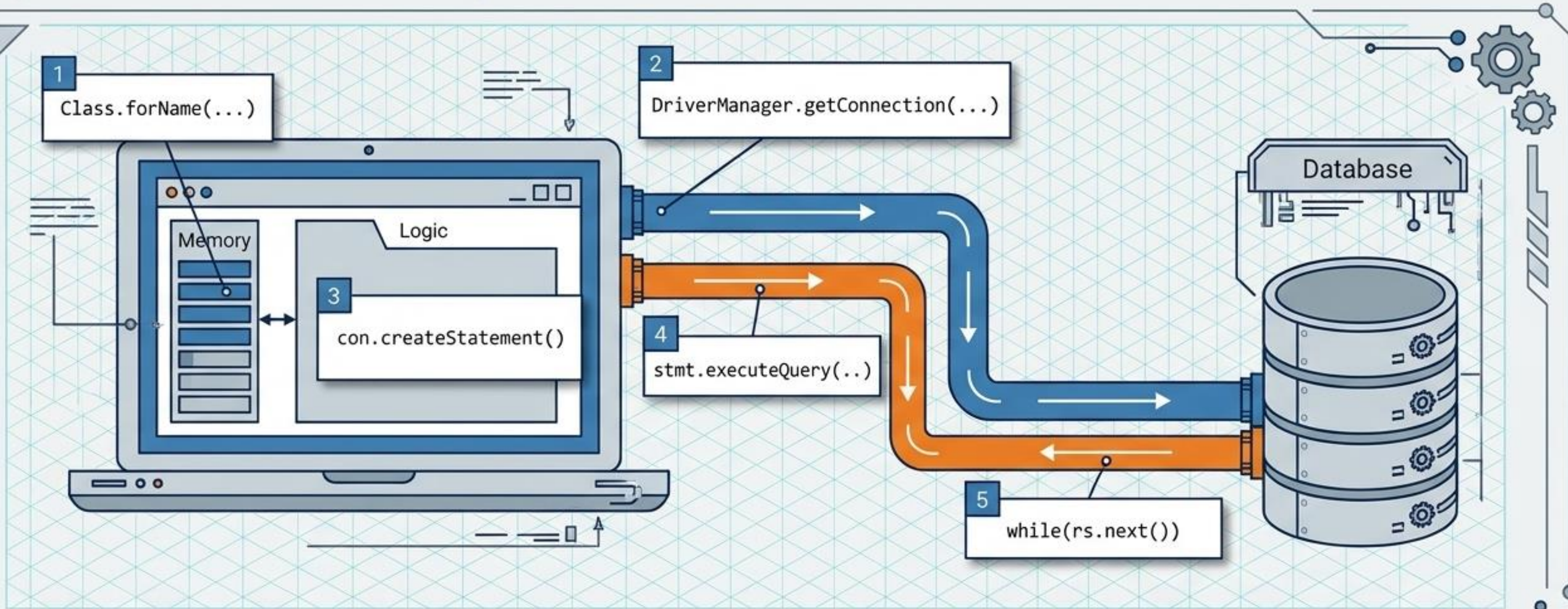
Database

aast.students ×							
#	Name	Datatype	Length/Set	Unsig...	Allow...	Zerofill	Default
1	registration	INT	11				
2	Name	VARCHAR	50				
3	Grade	CHAR	50	☐	☒	☐	U

4. Database locates and
returns matching row.

5. UI parses
ResultSet and
displays Grade "B"

Synthesis: Theory Meets Execution



The Takeaway: The abstract `java.sql` interfaces physically map to the rigid spatial realities of client-server engineering.

Production Checklist: JDBC Best Practices



Driver Management

Rely on JDBC 4.0 auto-loading; ensure the correct **Type 4 Thin Driver JAR** is physically present in the build classpath.



Security

Always utilize `PreparedStatement` rather than a standard `Statement` to structurally block SQL injection attacks.



Performance

Do not leave cursors hanging open. Implement `try-with-resources` blocks to guarantee `con.close()` execution under all error conditions.



Data Integrity

Ensure extraction methods (e.g., `getInt`, `getString`) strictly match the schema datatypes defined in the RDBMS.